OOPS | SYCSE Mini Project

UML Class Diagram Code Generator in C++

Mahek Shukla • Rohan Upadhye • Chirayu Agrawal • Yatharth Agrawal

Roll No: 7, 10, 12, 20 resp.

```
class Student {
private:
   string name;
   int rollNo;
public:
   string getName();
   void setName(string);
};

// auto-generated
```

Github Repository :
https://github.com/Mahek-Technophile/OOP-Class-Diagram-to-CPP-file-converter-.git

What Are We Building?

INPUT

```
CLASS Student
ATTR private string name
ATTR private int rollNo
METHOD public string getName()
METHOD public void setName(string)
END
```

diagram.txt

Generator

OUTPUT

```
class Student {
private:
    string name;
    int rollNo;
public:
    string getName();
    void setName(string);
};
```

Student.h (auto-generated)

System Architecture

1

Input Parser

Reads diagram.txt
line by line

2

UML Model Classes

UMLAttribute
UMLMethod
UMLClass

3

Code Generator

Abstract base class
CPPCodeGenerator

4

File Writer

Writes compilable
.h files to output/

UML Model Classes — The OOP Backbone

UMLAttribute

Fields / Methods:

```
string access
string type
string name
```

e.g. "private string name"

UMLMethod

Fields / Methods:

```
string access
string returnType
string name
vector<string> params
```

e.g. "public void setName(string)"

UMLClass

Fields / Methods:

```
string className
vector<UMLAttribute> attrs
vector<UMLMethod> methods
void addAttribute()
void addMethod()
```

Main container — most important!

OOP Pillars Applied

Constructors

1. UMLAttribute, UMLMethod & UMLClass each define a constructor
2. Takes access, type/returnType & name as parameters
3. Uses this-> to assign values to member variables
4. Called automatically when an object is instantiated

File Operations

1. CPPCodeGenerator opens an ofstream to output/<ClassName>.h
2. Checks is_open() before writing to catch missing folders
3. Writes class skeleton — private fields then public members
4. Closes the stream with outFile.close() after generation

Operator Overloading

1. operator+ defined twice inside UMLClass
2. Right-hand type (UMLAttribute vs UMLMethod) decides the branch
3. Returns *this so calls can be chained with =
4. Example: cls = cls + UMLAttribute(...) / cls + UMLMethod(...)

Function Overloading

1. operator+ is overloaded twice-once taking a UMLAttribute, once taking a UMLMethod
2. The Compiler distinguishes between the two purely by the type of right hand operand.
3. Both versions return *this, allowing the result to be assigned or chained

Code in Action

UMLAttribute.h — Constructor

```
class UMLAttribute {
public:
    string access, type, name;

    UMLAttribute(string access,
                 string type,
                 string name) {
        this->access = access;
        this->type   = type;
        this->name   = name;
    }
};
```

CppCodeGenerator.h — File Ops (fstream)

```
string filename = "output/"
                + cls.className + ".h";
ofstream outFile(filename);

if (!outFile.is_open()) {
    cout << "ERROR: file not found";
    return;
}

outFile << "class " << cls.className
        << " {" << endl;
outFile.close();
```

UMLClass.h — Operator Overloading & Function Overloading

```
// operator+ for UMLAttribute
UMLClass operator+(const UMLAttribute& a) {
    attributes.push_back(a);
    return *this;
}

// operator+ for UMLMethod
UMLClass operator+(const UMLMethod& m) {
    methods.push_back(m);
    return *this;
}

// Usage in main.cpp:
cls = cls + UMLAttribute(...);
```

Step 3 — The Input Parser

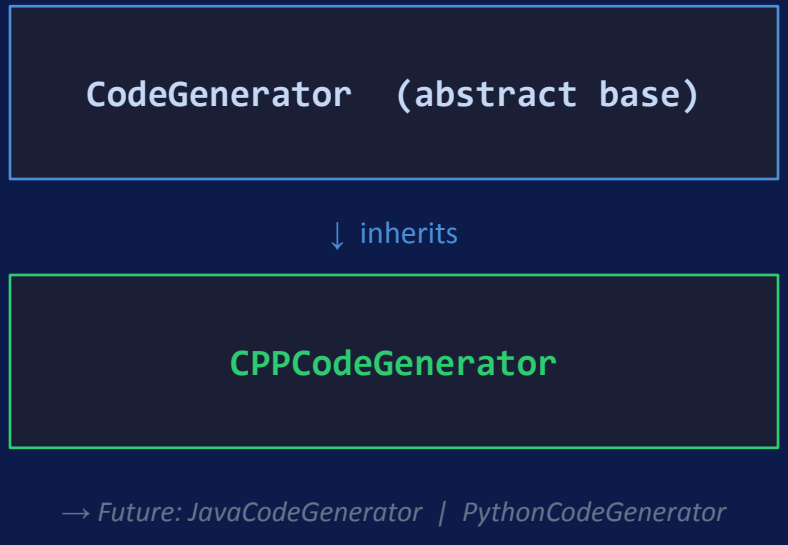
How it works:

1. **Open file**
   ifstream reads diagram.txt
2. **Read line by line**
   getline() processes each row
3. **Parse keyword**
   Detects "CLASS", "ATTR", "METHOD", "END"
4. **Build objects**
   Creates UMLClass / UMLAttribute / UMLMethod
5. **Store in vector**
   vector<UMLClass> holds all classes

UMLParser.cpp (pseudocode)

```
open file("diagram.txt");
while (getline(file, line)) {
  if line starts "CLASS"
    → new UMLClass(name)
  if line starts "ATTR"
    → new UMLAttribute(...)
  if line starts "METHOD"
    → new UMLMethod(...)
  if line == "END"
    → save class to vector
}
return vector<UMLClass>;
```

Step 4 — The Code Generator



What CPPCodeGenerator writes:

```
write: "class Student {"    →  write private attributes
write: "public:"            →  write public methods
write: "};"                 →  close and save as Student.h
```

Project File Structure

```
/UML-Code-Generator

├── main.cpp
├── UMLParser.h
├── UMLParser.cpp
├── UMLClass.h
│   ├── Attribute.h
│   └── Method.h
├── CodeGenerator.h
├── CPPCodeGenerator.cpp
├── diagram.txt
└── output/
    └── Student.h
```

Key Files Explained

`main.cpp`
Entry point — wires parser + generator together

`UMLParser.cpp`
Reads diagram.txt, builds UMLClass objects

`UMLClass.h`
Main container with attribute & method vectors

`CodeGenerator.h`
Abstract base enabling polymorphism

`CPPCodeGenerator.cpp`
Implements generate() → writes .h files

`diagram.txt`
Your UML input text file

Demo Output — What It Produces

diagram.txt (INPUT)

```
CLASS Student
ATTR private string name
ATTR private int rollNo
ATTR private float marks
METHOD public string getName()
METHOD public void setName(string)
METHOD public int getRollNo()
END
```

→

Student.h (OUTPUT)

```
#ifndef STUDENT_H
#define STUDENT_H
#include <string>
using namespace std;

class Student {
private:
    string name;
    int rollNo;
    float marks;
public:
    string getName();
    void setName(string);
    int getRollNo();
};
#endif
```

The Team

07

Mahek Shukla

PRN: 1262241984

SYCSE | OOPS Mini Project

10

Rohan Upadhye

PRN: 1262241998

SYCSE | OOPS Mini Project

12

Chirayu Agrawal

PRN: 1262242010

SYCSE | OOPS Mini Project

20

Yatharth Agrawal

PRN: 1262242058

SYCSE | OOPS Mini Project